Aula 07 — Expressões, Atribuição e Controle de Fluxo

Nome: João Paulo Candido da Costa · **R.A:** 24000797-2

1. Panorama da aula

O tema central é entender exatamente o que um programa executa: como as expressões são calculadas, como os valores são guardados em variáveis e como a ordem de execução é desviada. Precedência mal lida, conversão automática de tipos e desvios de fluxo são as armadilhas mais comuns.

Três falhas famosas mostram que esses detalhes têm consequências reais:

- **goto fail (Apple, 2014):** um `if` sem chaves era seguido por duas linhas `goto fail;` idênticas. A segunda executava sempre, pulando a checagem da assinatura, e o iOS e o macOS passaram a aceitar certificados forjados.
- **Zune (Microsoft, 2008):** no último dia de um ano bissexto, um laço `while` deixava de avançar e rodava para sempre, travando os aparelhos.
- **PHP e o operador `==`:** a comparação `"0e123" == "0e456"` resulta verdadeira, pois as duas strings são lidas como 0 em notação científica. Sistemas que comparavam hashes assim aceitavam senhas erradas.

Em nenhum dos três casos a regra de negócio estava errada. A falha estava na avaliação de uma expressão ou numa estrutura de controle.

2. Expressões

2.1 Precedência e associatividade

A precedência diz qual operador é resolvido antes (multiplicação antes de soma). A associatividade desempata operadores de mesmo nível: `10 - 4 - 3` é lido como `(10 - 4) - 3`. A maioria agrupa da esquerda para a direita, mas a potência `**` agrupa pela direita em Python, JavaScript, Ruby e Fortran. Assim, `2 ** 3 ** 2` vale `2 ** 9`, ou 512.

Essas regras são escolhas de projeto. APL avalia tudo da direita para a esquerda (`3 × 4 + 5` dá 27). Lisp dispensa precedência porque escreve o operador antes dos operandos, entre parênteses: `(+ a (* b c))`.

Casos que confundem:

- `-2 ** 2` dá -4 em Python (a potência vem antes do sinal negativo), é erro de sintaxe em JavaScript e `=-2^2` dá 4 no Excel.
- Divisão com negativos: Java e C cortam em direção ao zero (`-7 / 2` dá -3); Python arredonda para baixo (`-7 // 2` dá -4).
- Na dúvida, parênteses resolvem.

Teste em Python:

```
a, b, c = 3, 4, 5
print(a + b * c)        # 23
print((a + b) * c)      # 35
print(-2 ** 2)          # -4
print((-2) ** 2)        # 4
print(2 ** 3 ** 2)      # 512
print(10 - 4 - 3)       # 3
print(-7 // 2, -7 % 2)  # -4 1
```

O ponto-chave é `-2 ** 2` resultar em -4, e não 4.

2.2 Ordem dos operandos e efeitos colaterais

Uma função tem efeito colateral quando modifica uma variável global ou um parâmetro. Nesse caso, a ordem em que os operandos são lidos altera o resultado. No exemplo do livro, `a = a + fun1()` com `a = 5`, e `fun1` muda `a` para 17 e retorna 3. Se `a` for lido antes da chamada, o resultado é 8; se depois, é 20.

```
#include <stdio.h>

int a = 5;

int fun1(void) {
    a = 17;
    return 3;
}

int main(void) {
    a = a + fun1();
    printf("a = %d\n", a); // gcc: a = 20
    return 0;
}
```

Nos testes, Java imprime 8, pois a linguagem garante avaliação da esquerda para a direita. C (gcc) e Go imprimem 20: o padrão de C não fixa a ordem, e Go não define quando a variável é lida em relação à chamada.

Transparência referencial: sem efeitos colaterais, a ordem deixa de importar e expressões de mesmo valor podem ser trocadas entre si (`f(x) + f(x)` equivale a `2 * f(x)`). Funções puras garantem essa propriedade.

2.3 Sobrecarga de operadores

Sobrecarregar é dar mais de um significado ao mesmo símbolo. Em Java, `+` soma inteiros, soma reais e junta textos: `'a' + 1` dá 98 (o caractere vira inteiro), enquanto `"" + 'a' + 1` dá `"a1"`.

C++, C#, Python, Rust e Kotlin permitem que o programador defina sobrecargas (em Python via `__add__`, em Rust via trait `Add`). Java não permite, e por isso `BigDecimal` exige `x.add(y)` no lugar de `x + y`. A sobrecarga deixa o código mais legível (`v1 + v2`), mas pode ocultar custo e comportamento.

2.4 Conversões de tipo

Tipo de conversão	O que faz	Risco	Exemplo
Alargamento	Vai para um tipo maior (int → double)	Perda de precisão	`123456789` → float → int vira `123456792`
Estreitamento	Vai para um tipo menor	Perda de informação	`(int) 3.99` dá 3; `(byte) 200` dá -56
Coerção	Conversão implícita, feita pelo compilador	Esconde erros de digitação	`"5" - 2` dá 3 em JS
Cast	Conversão explícita, pedida pelo programador	Fica visível no código	`(double) x`

Como a coerção pode mascarar erros, Go, Rust, ML e F# não convertem tipos numéricos automaticamente.

JavaScript e PHP são muito agressivos na coerção. Em JS, `"5" + 2` dá `"52"`, `"b" + "a" + +"a" + "a"` dá `"baNaNa"` e o `==` não é transitivo: `0 == ""` e `0 == "0"` são verdadeiros, mas `"" == "0"` é falso. Regra prática: em JS e PHP, prefira `===` e `!==`, que comparam sem converter.

```
console.log("5" - 2);                      // 3
console.log("5" + 2);                      // 52
console.log("5" * "2");                    // 10
console.log(true + 1);                     // 2
console.log(null + 1, undefined + 1);      // 1 NaN
console.log("b" + "a" + +"a" + "a");       // baNaNa
console.log(0 == "", 0 == "0", "" == "0"); // true true false
console.log(0 === "", 0 === "0");          // false false
```

2.5 Expressões relacionais, booleanas e curto-circuito

C não tem um tipo booleano de verdade: 0 é falso e todo o resto é verdadeiro. Por isso `3 > 2 > 1` dá 0 em C (`3 > 2` vale 1, e `1 > 1` é falso). Python encadeia comparações, então a mesma expressão dá `True`.

Python e JavaScript aceitam qualquer valor numa condição (0, vazio e nulo valem como falso). Java, C#, Go e Rust exigem um booleano.

No curto-circuito, o resultado é decidido antes de avaliar todos os operandos. É o que torna seguro `while (i < lista.length && lista[i] != chave)`: se o índice estourar, a segunda parte nem é lida. Em Java, `&&` faz curto-circuito e `&` não. Atenção: um efeito colateral no segundo operando pode nunca acontecer.

Em Python, `and` e `or` retornam um dos operandos: `0 or "padrão"` dá `"padrão"`. Em JavaScript, `quantidade || 10` descarta o 0 legítimo; `??` corrige isso, pois só substitui `null` e `undefined`.

3. Atribuição

Em C e JavaScript, atribuir é uma expressão: a operação devolve o valor que foi guardado. Isso permite escrever `sum = count = 0` ou `while ((ch = getchar()) != EOF)`. O preço é que `if (x = y)` compila e atribui em vez de comparar.

Java e C# bloqueiam esse erro porque o `if` só aceita booleano. Em Python, Go e Rust, a atribuição é uma sentença, não uma expressão (Python oferece apenas o operador "morsa" `:=` para esse uso).

Outras formas de atribuir:

- Composta: `a += b`.
- Unária: `count++`.
- Múltipla: `a, b = b, a` (Python e Go).
- Ligação imutável: em linguagens funcionais e em Rust, `let` associa um nome a um valor fixo; `let x = x + 1` cria uma nova variável que encobre a anterior (sombreamento).

Atribuição de modo misto: C converte para qualquer lado sem avisar (`int i = 3.7` guarda 3; `char c = 300` guarda 44). Java só aceita alargamento sem cast, mas `i += 3.7` compila porque o `+=` embute um cast implícito.

Armadilha clássica: `double taxa = acertos / total * 100;` com operandos inteiros resulta em 0.0. A divisão inteira acontece primeiro, e só o resultado já truncado é convertido para `double` (é a questão 6 do exercício).

4. Estruturas de controle

4.1 Quantas estruturas são necessárias?

Böhm e Jacopini (1966) demonstraram que sequência, seleção e um laço lógico são suficientes para expressar qualquer algoritmo, sem precisar de `goto`. Estruturas como `for`, `switch`, `do` e `foreach` existem por conveniência. O limite é a legibilidade: quanto mais recursos uma linguagem tem, menor a parte do código alheio que cada pessoa reconhece.

4.2 Seleção de dois caminhos e o "else pendente"

Em Java, C, C++ e C#, o `else` se liga ao `if` mais próximo que ainda não tem `else`, e a indentação não conta (somente Python e F# a consideram). Em `if (sum == 0) if (count == 0) result = 0; else result = 1;`, o `else` pertence ao `if` interno, então `result` pode virar 1 mesmo que a indentação sugira o contrário.

Chaves obrigatórias (Perl, Go e Rust) eliminam essa ambiguidade. Em Rust o `if` é uma expressão que devolve valor; em Python há a forma `x if c else y`.

4.3 Seleção múltipla em quatro linguagens

O mesmo exemplo, com `n = 2`, foi escrito em C, Java, Go e Rust para comparar o que cada um imprime.

```
// C: sem break, o controle escorre para os casos seguintes
int n = 2;
switch (n) {
    case 1: printf("um\n");
    case 2: printf("dois\n");
    case 3: printf("tres\n");
    default: printf("outro\n");
}
// Saída: dois, tres, outro
```

```
// Java 14+: switch com -> é uma expressão e não escorre
int n = 2;
String r = switch (n) {
    case 1 -> "um";
    case 2 -> "dois";
    case 3 -> "tres";
    default -> "outro";
};
System.out.println(r); // dois
```

```
// Go: cada caso termina sozinho; escorrer exige fallthrough
n := 2
switch n {
case 1:
    fmt.Println("um")
case 2:
    fmt.Println("dois")
case 3:
    fmt.Println("tres")
default:
    fmt.Println("outro")
} // dois
```

```
// Rust: match precisa cobrir todos os casos
let n = 2;
let r = match n {
    1 => "um",
    2 => "dois",
    3 => "tres",
    _ => "outro", // sem esta linha, não compila
};
println!("{}", r); // dois
```

Linguagem	Saída com n = 2	Escorre entre casos?	Caso padrão	Destaque
C	dois, tres, outro	Sim, se faltar `break`	Opcional	Fácil esquecer o `break`
Java (`->`)	dois	Não	Obrigatório para `int`; exaustivo para `enum`	Vira expressão com valor
Go	dois	Só com `fallthrough`	Opcional	Dispensa `break`
Rust (`match`)	dois	Não	Cobertura total obrigatória	Aceita faixas e padrões

C é a única das quatro em que escorrer é o comportamento padrão; o livro classifica esse `switch` como "relativamente primitivo", já que o `break` é um `goto` limitado. As demais eliminam essa classe de erro, e o `match` de Rust ainda obriga a tratar todos os casos.

4.4 Laços

No `for` de C, as três expressões são opcionais e a variável de controle pode ser alterada dentro do corpo. Por isso `for (int i = 0; i < 10; i++) { printf("%d ", i); i += 2; }` imprime `0 3 6 9`. Já o `for` de Python percorre um iterável, então alterar `i` no corpo não muda nada (imprime de 0 a 9).

Nos laços lógicos, o teste pode vir antes (`while`) ou depois (`do-while`, que roda pelo menos uma vez). Java e Perl têm `break` com rótulo, que encerra dois laços de uma vez. Em JavaScript, `for...in` percorre as chaves (strings) e `for...of` percorre os valores; somar com `for...in` produz `"0012"`.

Caso Zune: com `dias == 366` num ano bissexto, nenhum ramo do `while` modificava `dias`, e o laço nunca terminava. A pergunta certa é qual invariante o laço deveria garantir: a cada volta, `dias` precisa diminuir.

4.5 goto e comandos protegidos

O `goto` é a instrução de controle mais poderosa e, por isso, a mais arriscada: torna o programa difícil de ler e de manter (Dijkstra, 1968, "Go To Statement Considered Harmful"). Java não tem `goto`; C, C++, C# e Go têm. `break`, `continue` e `return` funcionam como desvios restritos e melhoram a legibilidade.

No *goto fail* da Apple, a segunda linha `goto fail;` estava fora do `if` por falta de chaves. A verificação da assinatura nunca era executada e `err` permanecia 0, valor que significa "válido". Chaves obrigatórias (Go e Rust) ou o aviso de indentação enganosa do `gcc -Wall` teriam evitado a falha.

Comandos protegidos de Dijkstra: em `if ... fi` e `do ... od`, todas as guardas são avaliadas. Se mais de uma for verdadeira, uma delas é escolhida de forma não determinística; se nenhuma for, o `if` gera erro e o `do` termina. Essa ideia fundamenta o CSP de Hoare e o `select` de Go, que sorteia entre canais prontos. Em Haskell, as guardas são testadas em ordem, sendo portanto determinísticas.

5. Exercício "Encontre o bug"

Para cada trecho, responde-se: (a) o que é impresso, (b) qual regra da linguagem gera o problema e (c) como corrigir. As respostas foram confirmadas executando o código.

Questão 1 (JavaScript)

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos) total += p;
console.log("Total: " + total);
```

- **(a) Saída:** `Total: 0012`.
- **(b) Causa:** `for...in` itera sobre as chaves do array, que são as strings `"0"`, `"1"` e `"2"`, e o `+` passa a concatenar.
- **(c) Correção:** usar `for...of` ou `precos.reduce((a, b) => a + b, 0)`.

Questão 2 (Python)

```
def desconto(preco, d=None):
    d = d or 10         # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

- **(a) Saída:** `180.0`, quando o esperado era `200.0` (desconto zero).
- **(b) Causa:** em `d or 10`, o 0 é tratado como falso e o `or` devolve o segundo operando.
- **(c) Correção:** `if d is None: d = 10`. Em JavaScript, o `??` resolveria.

Questão 3 (Java)

```
int dia = 2;
String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

- **(a) Saída:** `inválido`.
- **(b) Causa:** sem `break`, a execução escorre do `case 2` até o `default`, sobrescrevendo o valor.
- **(c) Correção:** colocar `break` em cada caso ou usar o `switch` com `->` (Java 14+).

Questão 4 (C)

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

- **(a) Saída:** `saldo insuficiente` e `fim`, embora o saque seja menor que o saldo.
- **(b) Causa:** o `;` logo após o `if` é uma sentença vazia; o `printf` seguinte fica fora da condição.
- **(c) Correção:** usar chaves sempre. O `gcc -Wall` alerta com `-Wempty-body`.

Questão 5 (Go)

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

- **(a) Saída:** `dentro: 20` e `fora: 10`.
- **(b) Causa:** o `:=` dentro do bloco declara um novo `x` (sombreamento) em vez de alterar o de fora.
- **(c) Correção:** trocar por `x = x * 2`.

Questão 6 (Java)

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

- **(a) Saída:** `0.0%`.

- **(b) Causa:** `7 / 10` é divisão inteira e resulta em 0; só depois o valor é convertido para `double` (atribuição de modo misto).
- **(c) Correção:** `acertos * 100.0 / total` ou `(double) acertos / total * 100`, ambos resultando em `70.0%`.

6. Pesquisa complementar

- **Expressões em Ruby:** operadores são métodos (`1 + 2` equivale a `1.+(2)`) e podem ser redefinidos. Como em Python, `**` associa pela direita. Iteradores recebem blocos: `3.times { |i| print i }` imprime `012`.
- **Expressões em Lisp:** a notação prefixa e os parênteses definem a ordem, então não há precedência. `(+ a (* b c))` corresponde a `a + b * c`, e `(+ 1 2 3 4 5)` mostra um operador aplicado a vários operandos. O `if` também devolve valor: `(if (> a 0) a (* 2 a))`.
- **Expressões condicionais:** escolhem um valor dentro de uma expressão. Em C e Java há o ternário `c ? a : b`; em Python, `x if c else y`; em Rust e F#, o próprio `if` retorna valor.
- **Erros em expressões:** segundo o livro (seção 7.4.3, p. 313), surgem dos limites da aritmética da máquina, como estouro (overflow) e divisão por zero. A linguagem pode lançar exceções ou ignorar o problema silenciosamente.
- **Tabelas de desvio:** num `switch` com muitos casos, o compilador pode gerar uma tabela de saltos (acesso direto ao caso) ou uma busca binária, em vez de testar caso a caso. Por isso um `switch` grande tende a ser mais rápido que uma sequência de `if`.
- **Laços em linguagens funcionais:** sem variáveis mutáveis, a repetição é feita por recursão.
- **Iteradores e blocos de Ruby:** em vez de um `for` com contador, o objeto recebe um bloco e o executa para cada elemento (`each`, `times`, `select`), na mesma linha do `forEach` de JavaScript.
- **Dijkstra (1968) e Knuth (1974):** o artigo de Dijkstra argumenta que o `goto` dificulta entender o programa e provar que ele está correto. Knuth respondeu que alguns usos são aceitáveis, como saltar para um bloco único de limpeza no tratamento de erros em C.

7. Síntese

Tema	Armadilha	O que ajuda
Precedência	`-2 ** 2`, `1 + 2 + "3"`	Parênteses; JS exige `(-2) ** 2`
Ordem dos operandos	`a + f()` com efeito colateral dá resultados diferentes em C e Go	Java fixa a ordem; funções puras
Conversões	Coerção silenciosa em C, JS e PHP	Conversão explícita (Go, Rust); `===` em JS
Booleanos	0, `""` e `[]` como falsos; `a > b > c` em C	`if` só com booleano (Java, Go, Rust); `??`
Atribuição	`if (x = y)` compila em C	Atribuição como sentença (Python, Go, Rust)
Seleção	`switch` sem `break`; else pendente	`->` em Java, `match` exaustivo, chaves obrigatórias
Laços	`for...in` em JS; laço que não avança	`for...of`, iteradores, `break` com rótulo
Desvio	`goto` irrestrito	`break`, `continue` e `return`

Linguagens mais recentes (Go, Rust, Kotlin, Swift) incorporaram as lições de C: booleano obrigatório no `if`, chaves obrigatórias, conversões explícitas, seleção múltipla sem escorrer e atribuição como sentença. Cada restrição elimina uma família inteira de bugs, trocando um pouco de facilidade de escrita por confiabilidade, o trade-off apresentado no Capítulo 1.

Referências

- SEBESTA, R. W. *Conceitos de Linguagens de Programação*. 11ª ed. Capítulos 7 e 8 (p. 299–361).
- Slides e exemplos da Aula 07.